

From guidelines to culturally competent artificial intelligence

Antonio Sgorbissa

DIBRIS Department, Università degli Studi di Genova, Genova, Italy

LEARNING OBJECTIVES

- To become aware of some of the most popular knowledge representation techniques, aiming to represent knowledge in a formal language that an Artificial Intelligence (AI) program can process.
- To understand the fundamental mechanisms to store knowledge using computer ontologies and perform probabilistic reasoning about known and unknown facts using Bayesian networks.
- To understand how cultural competence can be embedded into an AI program that encodes guidelines provided by transcultural nursing experts, using the CARESSES system as a case study.
- To understand the fundamental mechanisms to make the robot interact verbally and nonverbally with people by considering cultural differences and learning personal preferences to avoid stereotyped representations.

8.1 Introduction

“Does the robot know how to do this?” “The robot knows that I am doing that.” “The robot doesn’t know how to do that; therefore, it is searching on the Internet for it.”

Knowledge plays a crucial role in all activities of our life. We may know or not know how to do things, we may know or not know how to answer a question, we may know or not know how to deal with people. Everything that implies an intentional act relies on some sort of knowledge. According to the online Oxford Dictionary, knowledge is “the information, understanding, and skills that you gain through education or experience.”

Given these premises, we start by describing two of the most popular technologies commonly used in Artificial Intelligence (AI) for knowledge representation: Ontologies (Gruber, 1995) and Bayesian Networks (Pearl, 2000). Here, we will explain the basic principles of how they work. In the following sections, we will discuss how these technologies can provide the basis to embed a social robot with the required cultural knowledge to behave culturally competently according to the guidelines presented in previous Chapters 6 and 7.

The reader may be surprised that we are not mentioning Neural Networks in this chapter, given society's great expectations for Deep Learning applications (Schmidhuber, 2015). This choice is arbitrary and follows the motivations that have been presented in the last part of Chapter 2 and the fact that the CARESSES project will be presented as a case study to clarify key concepts. Due to the limited space, we will emphasize the technological solutions adopted in CARESSES.

8.2 Representing knowledge

8.2.1 Ontologies from parmenides to artificial intelligence

We said that knowledge plays a crucial role in doing things: knowing the correct phone number is a mandatory requirement for making a phone call, but also activities such as walking, picking an object from the floor, or riding a bicycle require knowledge. The difference between “knowing phone numbers” and “knowing how to ride a bike” is that phone numbers are expressed in “symbolic form”: knowledge of this kind can be easily stored and communicated by using symbols, digits in this case. Conversely, the knowledge required to ride a bike cannot be easily described using symbols. In principle, we might record the exact movements of all the joints of a person riding a bike, all the forces and momentum applied versus time, etc., and transform them in a sequence of numbers to be stored somewhere. Motion capture systems adopt this approach for creating Computer Generated Imagery (CGI) characters in movies. For instance, Gollum in “The Lord of the Rings” movie series was “created” by placing markers all over the body of the actor Andy Serkis and recording his exact movements, later processed by software for CGI animation. However, this works only with movies, since the detailed description of all the movements performed by a cyclist is not the knowledge used by the cyclist to ride his or her bicycle: nobody rides a bike by replicating prestored movements in a cyclic pattern. Not even a robot might work in that way: any minimum error or environmental change, for example, a slippery road or a sudden gust of wind, would likely produce a fall if a more robust mechanism to generate in real time the right movements that may restore the balance is absent. Finally, notice that, even in the case of motion capture for CGI animation, the sequence of numbers that we are using to describe the actors' movements is just an approximation of their actual movements in the real world. When we describe some aspect of the world using numbers or other symbols and mathematical or logical relationships between them, we say that we create a “model” of that particular aspect of the world. If the knowledge to be stored is a list of phone numbers, the situation is different: phone numbers are already composed of symbols. We might say that, in this case, the model to represent phone numbers corresponds exactly to actual phone numbers as they are in the real world.

It is not a surprise that there are not many situations as easy as modeling phone numbers. Typically, building a world model is easy when describing “codes” that humans have invented: car plates, passport IDs, people names, brands, etc., are intrinsically symbolic. However, in most situations, knowledge is not already available in symbolic form, and an effort to build a model of what we know is almost always required. The way to develop models of the world has been one of the preferred subjects of investigation in philosophy since its birth: later, it has become one of the most crucial research areas in AI and, more generally, Computer Science. Starting from the Greek philosopher Parmenides (515 BCE–450 BCE), many philosophers addressed the problem of defining “what is” (the Greek word, *ὄν*—being or existence, in Latin characters “on”). After that, philosophers moved to the definition of the possible categories of “being” and their attributes, a problem that the Greek philosopher Aristotle (384 BCE–322 BCE) addressed with the true spirit of a “knowledge engineer.” Notice that the concept of “being” or, in Ancient Greek, “on” is at the basis of the term “Ontology,” the branch of philosophy that studies reality: what exists, what are the attributes of things that exist, and the mutual relations between them. The word Ontology was first used by the German philosopher and pedagogue Jacob Lorhard (1561–1609) in his work *Ogdoas Scholastica* (1606), even if the interest for these studies dates back to Parmenides. More relevant for the scope of this book, the term Ontology has recently become popular in Computer Science; however, computer scientists use this term not to refer to the philosophical study of what exists and its categories but to a computer-understandable model of the world that is appropriate to describe the relevant aspects for a given application. Ontology is a countable noun for a computer scientist: you will likely hear computer scientists talking about “an Ontology,” meaning some formal representation that they can use to store the knowledge required by an AI program to work.

These concepts may be better explained through an example. Think about the room in which you are now (if you are in a room, otherwise please imagine it): there may be chairs, a table or a desk, walls ... or a bed if you are in a bedroom. The table can be of wood, plastic, or metal, and the same is probably true for your chair. The table has probably four legs unless you are sitting in a café in Paris, whereas the chair may have four legs or wheels, in case it is an office chair. These are just some of the things that exist in the small part of the world you are in now. The right way to describe knowledge of this kind is one of the problems that the branch of philosophy called Ontology may address; a formal description of the things around you and their mutual relationships would be called an Ontology by Computer Scientists.

But how is a computer Ontology built?

For instance, one may start by considering that the table I have in my kitchen is only one of the possible tables: other kinds of tables are office tables, café tables, séance tables, etc. All tables share some common properties related to their shape and, even more, to their functionality: however, they are also different from each other, for instance, in their dimensions and weight. Suppose I think about a “kitchen table”: a different mental image appears in my mind than a “café table,” and this mental image will likely impact my plans to look for a friend to help me move the table from one place to another place. An excellent way of representing this idea is to assume a general concept called “table,” and then define “kitchen table,” “café table,” and “séance table” as particular cases, which we will also refer to as “children” or “subconcepts” of “table,” [Fig. 8.1](#). It shall be pretty straightforward to replicate this process for all objects in the room, for instance, chairs: for sure, there are different kinds of chairs, and all of them can be represented as children of the generic concept “chair.” Please notice that the drawing in [Fig. 8.1](#) is

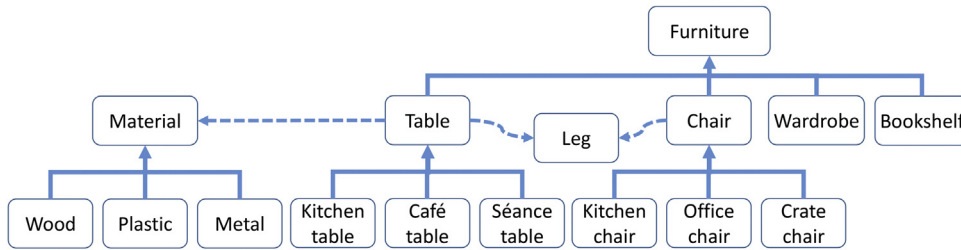


FIGURE 8.1 A computer ontology representing concepts and relationships between concepts.

not a flowchart, and therefore it is entirely different from the sketches that have been shown in [Chapter 2](#) to represent computer programs. In Ontologies, there is no flow of execution since there is no computer program to be executed: an Ontology is a static representation showing what exists in a given domain and mutual relationships between things that exist.

We can say something more about chairs. Suppose we focus on the functionalities of chairs, that is, the fact that they afford people to sit on them, and not on the fact that chair factories produced them: then, even a crate can be a chair. Only a few guests would complain if they are asked to sit on a crate, perhaps in a postindustrial style apartment, because a crate used as a chair is definitely a chair, isn't it? There may be some discussion among philosophers and knowledge engineers regarding this claim. Assessing the most appropriate position of each concept in the taxonomy is an arduous task, and some relationships may be less intuitive than others (or be different in different cultures, thus making the whole process more complex). However, we are confident that the idea of considering a "Crate chair" as a particular kind of chair may not look that absurd. Finally, one could observe that both a "Table" and a "Chair" are nothing more than "Furniture," a characteristic they share with a "Wardrobe" or a "Bookshelf," [Fig. 8.1](#).

Parent–child relationships (also referred to as "is-a" relationships, as they allow to express the concept "a table is-a type of furniture") are not the only possibility: for instance, one may observe that both tables and chairs have legs. This observation may lead to the idea of representing in the Ontology the concept "Leg," and then establishing a relationship between "Table" and "Chair" on the one side, and "Leg" on the other side: in this case, the relationship is not between a concept and its children, but rather it describes a property of tables and chairs, that is, the property of "having legs." Property relationships, differently from parent–child relationships, have not a unique name or meaning: the same concept may have heterogeneous properties such as, in the case of a table, the fact of having legs, the material it is made of, its dimensions, or its weight. Despite their different meaning, it is common to call all these properties using the prefix "has-N," where "N" is an arbitrary number. We will likely say that "Table" has the property "has-1-material" that relates it to the concept "Material," which may then be specialized into different kinds of materials; a property "has-4-leg" that links "Table" to the concept "Leg," and so on. In [Fig. 8.1](#), "is-a" parent–child relationships are shown as thick arrows; "has-N" properties are shown as dashed arrows.

This is almost everything we need for now. The knowledge encoded in the Ontology might also be written using a different, nongraphic formalism. For instance, we might use a logic language called Description Logics ([Baader et al., 2007](#)), its implementation called Ontology Web Language OWL2 ([Grau et al., 2008](#)), or its equivalent language used by Protégé, a popular

tool for the design of OWL2 Ontologies that we welcome you to try¹. Sentences look different in these languages, but they convey the same meaning. Very importantly, once we have created an Ontology as the one in Fig. 8.1 or its text-based equivalent, we may claim that we have created a “vocabulary,” which defines concepts based on other concepts: a description in this vocabulary might sound like “a table is a kind of furniture which has a tabletop, legs, a material, a dimension and a weight” (some of these elements are not shown in the figure). Researchers refer to this vocabulary as the Terminological Box (or TBox) of the Ontology.

Second, it is crucial to understand the implications of the taxonomy of concepts represented in Fig. 8.1. If a parent–child relationship exists in the Ontology between two concepts, it is assumed that the child concept inherits all properties of the parent concept. Think about the concepts “Table” (the parent) and “Café table” (the child): “Café table” is obviously made of some material and has legs, and this follows the fact that a “Café table” is indeed a “Table,” which is characterized by having those properties. Something exciting happens when encoding knowledge using an Ontology language: the formalism does not need to explicitly replicate for the child all the properties that have been declared for the parent since it is implicitly assumed that the child will inherit all the properties. This tremendously increases the expressiveness of the Ontology.

Third, Ontologies are typically designed by hand by humans: by keeping on with the same formalism we just used for tables and chairs, we might jokingly say that the design of Ontologies requires a particular child of the “Human” concept known as “Computer scientist” or, better, “Knowledge Engineer.” Recently, research has explored the possibility of automatically creating Ontologies starting from text written in natural language, for example, books, websites, or social media (Cimiano, 2006). However, knowledge learning is quite a complex task to be automated because it ideally requires a computer program to understand the meaning of the text to be extracted. As such, we will ignore automatic Ontology learning by suggesting that, in our view, humans are the ones that shall translate knowledge written in natural language, that is, the language they use to communicate, to the language used by computer Ontologies. Since, differently from natural language, Ontologies are written in a formal language without ambiguities, they can be easily processed and understood by a robot, which can use the knowledge stored therein to do valuable things. What makes this idea appealing is that the result will be easily understandable both by humans and robots.

Fourth, it is essential to remember that Ontologies, like any description made in any language, are just a model of reality and not reality. We will not venture into a debate about the “true nature of things”: still, we have to admit that we feel very uncomfortable to claim that there exist things that can be unambiguously classified as chairs or tables out there in the real world. Instead, we think that such words as “chair” or “table” are required by humans as basic bricks for communication and problem solving, but they are once again the results of a “modeling” process, that is, aimed at providing a simplified description that can be used to talk about the world. Without emphasizing too much the philosophical side of it, the example of a crate used as a chair may help capture what we are saying. Assigning univocal labels to things in the real world is not always straightforward, precisely because all these labels are an invention of humans to simplify their own lives, but things in the world do not have unambiguous labels in most cases. Computer scientists and knowledge engineers

¹ <https://protege.stanford.edu/>

face this kind of problem every day when trying to encode knowledge into Ontologies, an arduous work requiring a lot of thinking to properly organize things.

8.2.2 Probabilistic knowledge from reverend Bayes to Bayesian networks

Ontologies are not the only available tool to encode knowledge in a way understandable both by humans and by computer programs (or robots). The portfolio of possible tools and languages is vast, each having its peculiarities in terms of expressiveness (i.e., what kind of knowledge can I encode?), reasoning (i.e., what kind of hidden knowledge can I infer starting from the knowledge that was explicitly encoded in the system), and ease of use (i.e., what is the process to encode knowledge, and to what extent can this process be automated?).

According to the classification above, it is a well-known characteristic of Ontologies that they do not deal with knowledge expressed in probabilistic form. Since standard Ontologies come from formal logic, they tend to be quite “Manichaeian”: things are either true or false². Please remember that formal logic was born to make such inferences as “Socrates is a man; all men are mortal; therefore, Socrates is mortal.” In standard Ontology languages, it is not possible to say something like “a crate in the living room of a post-modern apartment has an 80% chance of being a chair.” Either we say crates are chairs, or we say they are not: there is no possibility for “it might be,” and it is not allowed to assign a numerical probability for an assertion to be true or false. As an aside, please notice that we said “80% chance” instead of “80% probability” because probabilities are typically expressed as values in the range from 0 to 1, where one means that a fact is undoubtedly true and zero means that a fact is surely false. Values in-between zero and one represent different levels of probability: when you flip a coin, you may say you have almost a 0.5 probability that the head comes out (we say “almost” because, once again, we are making an approximate model of how things work).

As the reader can observe, due to the impossibility of talking about probabilities, Ontologies’ expressivity turns out to be quite limited if you need to represent uncertain knowledge: this might look like a minor limitation, but, unfortunately, it is a major one. Experience tells us that knowledge about the world is typically uncertain; it doesn’t happen very frequently that we are in the condition of making statements that are undoubtedly true under any possible situation. Somebody might argue that the physical world is uncertain and vastly unknown, but this is not a big problem when we talk about facts of everyday life: how often do you use probabilities when discussing with somebody during your day? This argument may look even stronger considering that Ontologies are used to encode our own definitions of concepts: the definition of “Table” should not require us to introduce any probabilistic information as far as we agree on its meaning. A table has legs, a top, and it is made of some material that may be wood, plastic, or even steel: do we really need to say “maybe ...” or “it has a 0.8 probability of ...”? Of course, the example we made earlier in this chapter about the crate used as a chair would benefit from a description using probabilities, but are we really capable of producing many examples like that one? Or, more reasonably, is this “crate-chair-thing” an exception to the more general case in which knowledge is well-defined and can be represented with neither ambiguity nor uncertainty? It is well known that, in the history of

² Manichaeism is an ancient religion born in the Middle East, that taught a view of the things of the universe as being either completely good or evil, with no option in the middle.

biology, the platypus put naturalists and epistemologists in a severe crisis because of the complexity of classifying it both as a mammal and an animal that lays eggs. But, come on, let's be realistic: how many "platypuses" are there in the world that we want to represent?

It turns out that there are many platypuses around us. Even worse, platypuses tend to happily prosper when we consider the fact that, in the world, there are many different cultures. For instance, "God" is understood by most people in the world, who will use other words in their own languages but would agree on this translation in English. Almost every human being can materialize a mental image of "God" when hearing this word, whether you believe or not in God's existence. But once you start asking people what "God" is, it is very likely that the properties associated with this concept will tend to vary depending on the area of the world in which you ask the question. And what about other, more pragmatic concepts such as "Marriage" and the relationships between "Marriage" and the concept of "Family," "Parent," "Child," "House," "Sex," "Wedding party"? Do all people of the world materialize the same mental image of what a "Wedding party" is? Or take something as simple and familiar as "Bread," or "Coffee," or "Tea": what does an Italian person think when hearing the word "Coffee"? What does an American person think? Suppose you ask an Italian or an American person to draw a "Coffee:" would you get the same picture?

We will come back to this discussion again. For the moment, it is sufficient to say that, when dealing with knowledge expressed in probabilistic form, a popular tool exists for representing such knowledge and reasoning about it: Bayesian Networks (Pearl, 2000), which owe their name to Thomas Bayes (1701–61), an English philosopher and mathematician who laid the foundation of modern probability theory. As Ontologies, Bayesian Networks require defining the most important concepts involved in a given domain. However, in this case, relationships between concepts do not correspond to "is-a" parent–child relationships or "has-N" property relationships but rather express causal relationships between facts.

Probably, the most famous example ever of a Bayesian Network is the one shown in Fig. 8.2, which you may likely find in many introductory handbooks or online tutorials. If

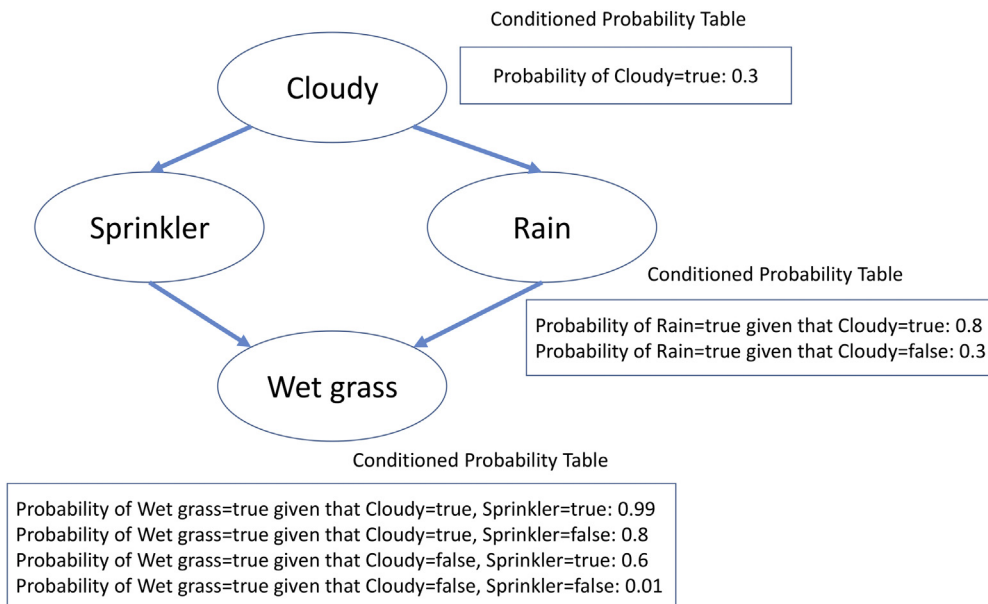


FIGURE 8.2 A Bayesian network expressing causal relationships between facts.

you want to have great fun, you may also download and try a software called Netica³ and try this and other examples by yourself. As you can easily see from the figure, four facts are involved in this network: these facts are the basic bricks to model the knowledge we have about that portion of reality we want to model. Who decides the basic bricks? As usual, this is a task for application domain experts, who choose the facts represented in the network and their mutual relationships. In this example, we want to represent the fact that the sky can be “Cloudy” or not, which impacts the fact that it may “Rain” or not. However, since we set up a smart “Sprinkler” in our garden, the fact that the sky is cloudy may also have an impact on the fact that the smart sprinkler will be automatically turned on today or not. Finally, the fact that it rained today (or not) and the fact that the sprinkler was turned on (or not) may have an impact on having “Wet grass” or not.

Experts will likely start by representing the fact that the sky can be “Cloudy” in a given day: this is modeled as a binary possibility if we assume that, at any given time, the sky can either be cloudy or not in a given place (whatever “Cloudy” means). Representing facts as binary possibilities is not mandatory in Bayesian Networks. In theory, we might consider that there are three (or more) possible situations concerning the sky: “Clean,” “Somehow cloudy,” “Very cloudy.” However, this does not change the following discussion and, therefore, we stick to the original description in which the sky can either be cloudy or not: we write the two possible events as “Cloudy = true” or “Cloudy = false.”

Imagine now that, at a given time, we do not know if the sky is “Cloudy” or not, for example, because we are imprisoned in a crate in the cellar. There are multiple usages for crates: this time, our host decided not to use it as a seat but to imprison undesired guests! Then, since we do not know the actual state of the sky because we cannot observe it, we may want to represent it with a probability value. For instance, after asking a weather expert, we might guess that the sky has a 0.3 probability for “Cloudy = true” and a 0.7 probability for “Cloudy = false.” This concept can be generalized: what about using the probabilities to represent, at any given time and not only in this very moment, the a priori knowledge we have about the sky being “Cloudy” or not “Cloudy”? What is the probability that tomorrow at 3 p.m. the sky will be “Cloudy”? What about next Monday at noon? This is precisely the purpose of the rectangle close to the “Cloudy” ellipse in Fig. 8.2: it represents the probability that, at any given time, the sky will be in one of the two possible states. Please notice that the rectangle contains only “Probability of Cloudy = true: 0.3” because one of the two mutually exclusive events must necessarily be true, and, therefore, the sum of the two probabilities is always 1: either “Cloudy = true” or “Cloudy = false,” which allows us to compute “Probability of Cloudy = false: 0.7”. The same happens when flipping a coin, throwing dice, or doing anything else: the probability of all mutually exclusive events must always be one since they cover 100% of the possibilities.

One might argue that the probabilities above are not easy to be computed, as they depend on many factors, including the season and the area of the world in which we are. We will come back soon to the problem of computing probabilities: for the moment, this consideration is helpful to introduce a crucial concept in Bayesian Networks, that is, the idea of “conditioned probability.” There is no doubt that things are related to each other and that the probabilities of some facts may depend on different facts. For instance, the probability that it rains in a given area of the world and a given season could be 0.2: however, if we

³ <https://www.norsys.com/netica.html>

may observe in advance that today is “Cloudy = true,” this probability will likely increase. More formally, we say that the probability of “Rain” is conditioned by “Cloudy,” and we introduce the concept “Probability of Rain given Cloudy.” The word “given” tells us that the second fact conditions the probability of the first fact.

There are a few essential concepts that the reader needs to remember.

First, and similarly to Ontologies, the Bayesian Network structure is typically designed by humans, who know the particular application domain they want to model and define the most important facts and how they relate to each other through cause–effect relationships. However, the probability values are not necessarily chosen by humans but can be automatically learned through observations. For instance, the probability that it will rain given that the sky is cloudy can be simply computed by considering a series of historical data (e.g., in the last year) and by calculating the ratio of times that it rained when the sky was cloudy: this is written as the “Probability of Rain = true given Cloudy = true.”

Second, and most importantly, we can use Bayesian Network to reason about facts that a robot observes or does not observe in the real world. Indeed, since the network encodes the knowledge about how a fact is affected by other facts, it can be used to collect information as time passes and draw conclusions about the probability of an unknown fact depending on the observations that have been done up to that moment. Consider the situation that we have no a priori information about the current state of things: we have not observed yet if it is cloudy or not, it rains or not, the sprinkler is on or off, and the grass is wet or not. Even if we do not know these facts for sure, we may know a priori probabilities: as already mentioned, it is sufficient that, during the last year, we recorded how many days the sky was cloudy or not, how many days it rained or not, etc., and we use the ratio of times that an event materialized as the basis to compute probabilities. But suppose now that, at some point, we can make some observation: for instance, from within our crate (we are prisoned in a crate in the cellar, do you remember?), we might hear the sprinkler turning on. This information is no more uncertain: in Bayesian Network terminology, “Sprinkler” is no more a “hidden fact” but an “observed fact.” We are now allowed to recompute all the probabilities in the network conditioned by the fact that the sprinkler is on, a piece of information that was not available before. For instance, we might expect the probability that the grass is wet to increase and the probability that the sky is cloudy to decrease, to mention two.

8.3 How to embed cultural competence into robots

8.3.1 Following guidelines to design artificial intelligence programs: knowledge representation without stereotypes

Cultural knowledge is one of the key elements at the core of cultural competence. As it has been already mentioned in [Chapters 1 and 5](#), cultural competence requires culture-awareness (i.e., being aware of the fact that culture permeates our own and other people lives), cultural knowledge (i.e., knowing the aspects of people’s lives on which culture has an impact), and cultural sensitivity (i.e., knowing how to interact with people appropriately). However, when implementing the three key elements of cultural competence into an intelligent system, keeping a clear distinction between them is pretty challenging. For the robot, culture-awareness is the direct

consequence of taking culture and cultural differences into account and encoding them through a proper formalism for knowledge representation. Cultural sensitivity is the consequence of knowing how to interact with people following guidelines for culturally competent interaction that have been defined by transcultural nursing experts and are, once again, encoded through formal tools for knowledge representation. Therefore, in the following, please keep in mind that cultural knowledge will not only include such concepts as “Table,” “Chair,” “God,” “coffee,” and how these concepts might be different in different cultures: instead, it will be extended to include the notion of how such knowledge should be used to interact with people, both verbally and nonverbally, up to the point of encompassing the whole idea of cultural competence.

The key idea to encode cultural knowledge in the system is to exploit the dualism that characterizes Ontologies, typically made of two components: the so-called Terminological Box (TBox) and Assertional Box (ABox). Notice that the connection between cultures and ontologies we propose takes inspiration from the so-called ontological turn that has produced an intense debate in anthropology in the last decades, starting from the seminal work “Thinking Through Things” by Henare, Holbraad, and Wastell ([Henare et al., 2006](#)). The interested reader is welcome to have a look at the report about the motion tabled at the 2008 meeting of the group for debates in anthropological theory at the University of Manchester, revolving around the question “Ontology Is Just Another Word for Culture?” ([Carrithers et al., 2010](#)).

The TBox is nothing more than what we have described in [Section 7.1](#): a definition of all the relevant concepts in a given domain, together with their mutual relationships. In the case of cultural knowledge, we may want to include not only tables and chairs but also concepts that are more evidently related to culture and cultural differences, such as beliefs, values, or habits. Very importantly, we will put together key concepts in all the cultures considered, possibly including side-by-side concepts that exist only in some cultures: the final vocabulary will be the union of all concepts. For example, after defining “Table,” we may want to create the children “Office Table,” “Kitchen Table,” “Séance Table,” “Café table.” But also, if needed, we might add “Chinese table” as a child of “Table,” or “Chinese Kitchen Table” as a child of “Kitchen Table” and so on. Or we may decide that the children of “Breakfast food” are “Scrambled egg,” “Coffee,” “Cappuccino,” “Tea,” “Miso soup,” “Natto,” “Ham”: to this end, we will add side-by-side typical breakfast food in the English, Italian, and Japanese cultures, some of which may be almost unknown to other cultures.

The ABox has never been mentioned before and needs now to be introduced. Unlike the TBox, the ABox is not used to describe concepts but “instances of concepts,” also known as “individuals” in Ontologies. To better understand the difference between the two, consider that TBox concepts are sometimes interpreted as “sets” or “classes” of things sharing the same properties: when we mention the concept “Table,” we are thinking about the set of all possible tables, that is, all objects that we would be happy to categorize as belonging to the same “Table” class. Things that have the “tableness” or, to put it with ancient Greek Philosophy, concepts corresponding to the Platonic ideal of “Table.” Following the same rationale, ABox instances shall be interpreted as specific elements of the set: “the table I have in my kitchen today May 24, 2021” or “the table I have in my office.” For example, after defining in the ABox the individual “The mug that my wife gave me as a birthday’s present,” the ABox is the right place to say that this very mug is on table I have in my office at this very moment.

The dualism between TBox and ABox is very useful in our case because it allows for distinguishing between “the world” (i.e., the TBox, describing the union of all concepts of interest in different cultures) and different “views of the world” (i.e., the ABox, that uses instances to describe how concepts in the TBox are perceived in different cultures). For example, think

about the concept of “Breakfast food” and its children: after being encoded as concepts in the TBox, they may be associated with a number N of different instances in the ABox that represent the different attitudes of N different cultures toward “Breakfast food.” What is “Breakfast food” for an Italian person? And for a Nigerian person? These multiple “views” of the same concept may simply be encoded through multiple ABox instances of “Breakfast food,” one for each different culture. Ideally, we can imagine that, for a given TBox, we will have a number N of ABoxes (we prefer to say “a number N of ABox layers since in each Ontology there should be one TBox and one ABox), one for each of the N cultures we want to represent. All instances belonging to a given ABox layer (say, the Italian ABox layer) express the view of Italian people about all concepts in the TBox. Fig. 8.3 shows this idea: green boxes represent instances corresponding to three different ABox layers, corresponding to the views of an English, Italian, or Japanese person of the same concept.

Before moving forward, we shall make two observations.

First, cultures are not countable: can we really draw the boundaries that define where a culture ends and another culture begins, allowing us to encode one culture in an ABox layer and the other in a different ABox layer? Does an “Italian culture” exist at all, or are there many different cultures whose boundaries are fuzzy? Is it true that the view of the world shared by people in one region of Italy will differ from another region, the view shared by people living in one town will differ from another town in the same region, the view of the world shared by people in one neighborhood will differ from another neighborhood in the same town? How many cultures do we need to describe in the Ontology to allow the robot to interact with people in a culturally competent way?

Second, if we push this rationale to its extreme consequences, isn’t it true that everyone is different from others? Perhaps we should focus on “personalization” in Robotics, whereas the concept of “cultural competence” may not be required at all? We know all too well that every attempt to associate a person to a culture incurs the risk of stereotyping. Taking culture into account may help the robot give more appropriate suggestions and behave more appropriately, but this works only in a probabilistic sense: guessing that an Italian person may like coffee or cappuccino for breakfast has more chance to succeed than assuming that the same person likes

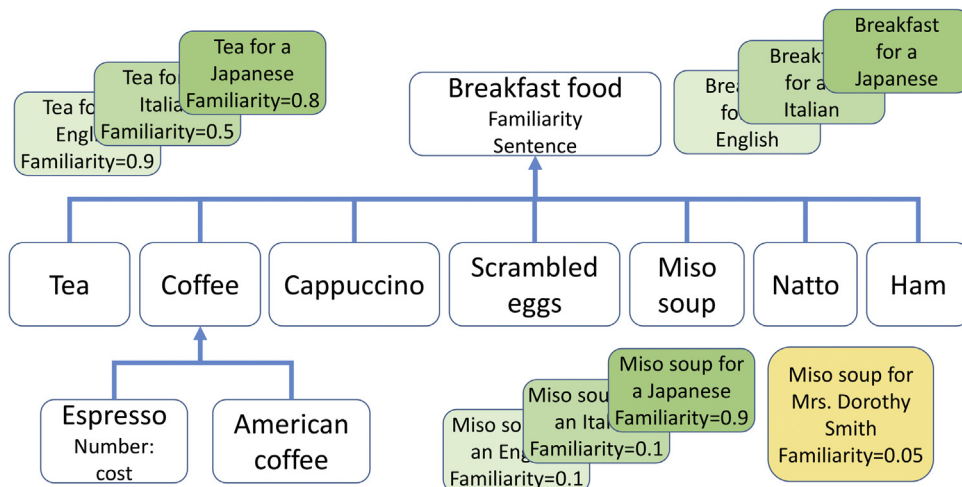


FIGURE 8.3 An ontology with concepts (white boxes) and instances (colored boxes).

having Japanese miso soup or natto, but this is not 100% safe. As discussed earlier in this chapter, associating a culture to a person corresponds to making a model of that person, which is approximate by its nature. When we make assumptions about what is most appropriate for a person depending on “how this person is expected to work,” we will necessarily fail sooner or later. Of course, there are different ways of building such a model, as discussed in the previous paragraph. For instance, we can choose to model Italians “as if” they shared a common view of the world, or we can choose to model people living in Genova “as if” they shared a common view. The second representation is a “higher resolution” way of stereotyping because there are more chances that people living in Genova will have some points in common, but it is still stereotyping. The only way to avoid stereotyping would be to develop a fully personalized system that “knows” the person and is specialized in interacting in the best possible way with this very person. As an aside, this is what typically happens with humans: knowing other people’s cultural background may initially help but, after we become close with somebody, knowing them becomes more relevant than knowing their cultural background. Unfortunately, adapting the robot behavior to a specific person requires a long time for learning: taking decisions based on the culture that the person identifies with, despite all limitations, provides a very efficient shortcut to start doing the right things as soon as possible.

In line with the observations above, please notice that using ABox layers to encode different “views” of the world, one is not constrained to represent cultures at the national level. The same mechanism can be used to represent the shared view of an ethnic group, people living in a region, or members of a family. There are no limitations on how this dualism between a unique TBox (the world) and multiple ABox layers (cultures) can be exploited. Then, why not using an ABox layer for each person, which might represent the particular view of the world that characterizes that person? This is precisely the solution we suggest: when a person interacts with the robot, a specific ABox shall be built that stores information about that person and only that person, coexisting with ABox layers representing different cultures. The ABox layer that encodes the individual preferences of Mrs. Dorothy Smith, an English woman, is shown in [Fig. 8.3](#) using yellow boxes.

We summarize the whole idea introduced so far as follows ([Bruno et al., 2019](#)).

- We call the knowledge encoded in the TBox layer “culture-generic” since it represents the knowledge that is not specific to any culture but is structured in such a way as to take into account the fact that different cultures exist. Here we represent all concepts belonging to multiple cultures as well as their mutual relationships: the concept “Espresso Coffee” coexists with “American Coffee” as children of “Coffee.”
- We call the knowledge encoded in the multiple ABox layers “culture-specific” since each ABox layer represents a different culture, encoding information about the attitude of that culture toward the corresponding concepts. For example, in the ABox corresponding to the Italian culture, the instances “Espresso Coffee for an Italian” and “American Coffee for an Italian” will include information about the attitude of Italian people toward “Espresso coffee” and “American coffee,” respectively, using a formalism that will be soon clarified.
- We call the knowledge encoded in the ABox layer corresponding to the person that is interacting with the robot “person-specific” since it includes information about the attitude of that person toward all concepts in the TBox. For example, instances “Espresso Coffee for Mrs. Dorothy Smith” and “American Coffee for Mrs. Dorothy Smith” will include information about the attitude of Mrs. Dorothy Smith, an English woman, toward “Espresso Coffee” and “American Coffee.”

But what is written precisely in the TBox and the ABox layers? What do we mean with “attitude toward a concept”? And how can we use this mechanism to avoid stereotyping, for example, to prevent the robot from postulating that all Italian people will have the same attitude toward espresso coffee (or toward miso soup, tables, chairs, God?). How can the guidelines for robotic cultural competence produced by transcultural nursing experts be implemented in the system?

Some additional information is required. Ontologies, as already mentioned, allow us to represent concepts, relationships between concepts, instances of concepts, and relationships between instances. Additionally, in the TBox, we can associate numerical properties to concepts (or even sequences of characters), and, analogously, we can assign values to these properties in the ABox. For instance, the concept “Espresso Coffee” might have numerical property expressing the fact that ordering an espresso in an Italian bar has a cost (what a surprise!) to be interpreted as a price in Euros. When we create an instance of “Espresso coffee” in the ABox, we can associate a specific number to that instance, representing the cost that espresso has in that particular bar at a given time.

In our case, we associate two main pieces of information to concepts in the culture-generic layer and then to instances of such concepts in the culture-specific layers:

- the familiarity that a given culture has with a concept (property “familiarity,” discussed in this section);
- the sentences that the system can use to talk about that concept when the latter is interpreted as a possible topic of conversation with the person (property “sentence,” discussed in the next section).

Please notice that, in [Fig. 8.3](#), the data properties “familiarity” and “sentence” are only shown in the block “Breakfast food.” However, as already said, they are inherited by all its children by exploiting the hierarchical structure of the Ontology.

The term “familiarity” shall be interpreted as a probability, and therefore, it can assume values in the range from 0 to 1. For example, consider the concept “Miso soup”: we need to represent the fact that people identifying with different cultures have a distinct familiarity with miso soup, not just meaning that they may know or not know what miso soup is, but also in the sense that they may have tried it or not, they may have miso soup more or less frequently, it may happen more or less regularly that somebody in their circle of friends or their family have a miso soup. Familiarity has all these meanings: the number associated with the instances of “Miso soup” for a given ABox layer, say the layer representing the English culture, corresponds to the probability that an English person will have familiarity with miso soup. Please notice that the familiarity value is stored in the culture-specific layer, but the information it encodes concerns individual persons: it is a guess, which may be wrong, that a randomly picked person selfidentifying with that culture may have familiarity with that concept. The “familiarity,” however, is not only stored in the culture-specific layer but also the person-specific layer of the ABox. Suppose again that the robot is interacting with Mrs. Dorothy Smith. In this case, the “familiarity” values in the person-specific layer describe whether Mrs. Dorothy Smith has or has not familiarity with having miso soup for breakfast.

The “familiarity” is the crucial element we propose to represent cultural knowledge in a nonstereotyped way, and may derive from the knowledge about different cultures provided by experts under the form of guidelines, see [Chapter 7](#) that introduced this concept. The robot, when interacting with a person, will have all the concepts in the Ontology at its disposal: even if it may be unlikely, an English person may love to have miso soup for

breakfast, something which is definitely more popular in Japan than in the UK, and we do not want to exclude this possibility. However, even if the robot has access to the entire Ontology, it will give priority to those concepts that are associated with a higher familiarity. For instance, if the robot needs to know what Mrs. Dorothy Smith usually has for breakfast, it will explore the possibility that she has tea before asking her if she has miso soup. This fact is reflected, in the figure, by “Tea” having a higher familiarity value than “Miso soup” in the culture-specific ABox Layer for the English Culture.

During the interaction, the system will acquire new information about the user. For instance, the robot may ask her if she ever tried miso soup even if it estimates that this is unlikely, and Mrs. Dorothy Smith may surprise the robot by positively replying that she indeed loves it. In this case, the information in the culture-specific layer will not change since nothing Mrs. Dorothy Smith says can modify our knowledge about the general attitude of English people toward miso soup: however, the “familiarity” value in the person-specific ABox layer of Mrs. Dorothy Smith will be updated accordingly. For example, if Mrs. Dorothy Smith declares she loves miso soup, the value will be updated to the maximum value, that is, 1. Similarly, suppose the robot poses a question about miso soup to Mrs. Dorothy Smith, and she replies that she does not know what the robot is talking about (something very likely for an older English person). In that case, the “familiarity” value will be updated to the minimum value, that is, 0. In principle, a “familiarity” value in-between 0 and one might be chosen if Mrs. Dorothy Smith declares that “well, I may know that miso thing ... but I am not that sure”, but we will not consider this possibility in the following.

Fig. 8.4 shows that this concept also concerns the actions that the robot can perform. The way to perform actions or, more generally, how to behave when interacting with people and with the environment may also be different depending on the cultural context. An example is greeting somebody: even if greeting by waving a hand is nowadays popular worldwide, greeting people with a bow is popular in Japan and other Asian countries, but not everywhere. Once again, actions and different ways of performing actions, or creating sequences of actions to achieve a given objective, are encoded as concepts in the culture-generic layer of the Ontology: instances in the culture-specific layers will then be assigned different “familiarity” values. As such, the instance of the concept “Bowing” in the Japanese ABox layer will have a higher familiarity value than the instance of “Bowing” in the Italian ABox layer, which does not exclude that Italian people may greet each other with a bow, but it makes it a less viable option with respects to different kinds of greetings.

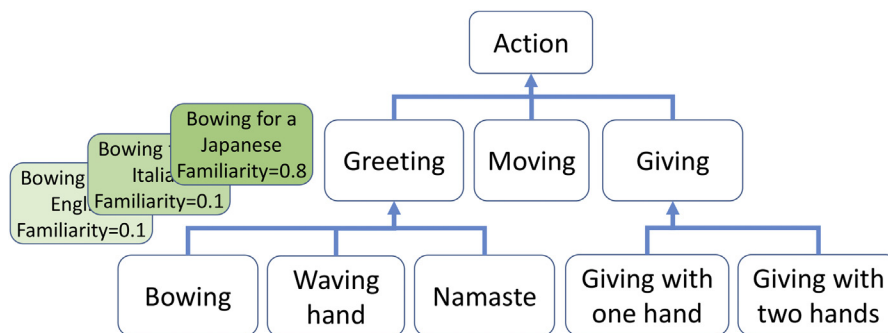


FIGURE 8.4 A section of the ontology representing robot actions.

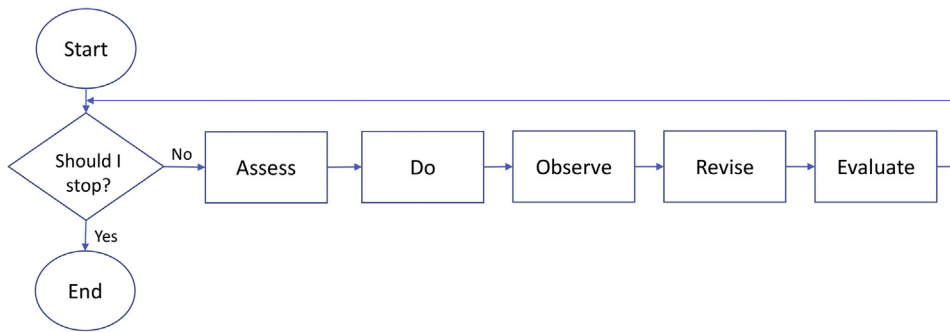


FIGURE 8.5 The flowchart of the ADORE model implemented as a computer program.

At this point, it shall be clear how the system may take advantage of cultural information and, at the same time, avoid stereotypes. Following [Chapter 7](#), the ADORE guidelines for culturally competent behavior can be implemented as follows:

- Assess the cultural identity of the person, which means choosing the appropriate culture-specific layer, that is, all instances in the corresponding ABox and the corresponding “familiarity” values;
- Do things with cultural sensitivity, which means choosing the topic of conversation and the actions of the robot that are associated with the highest “familiarity” values for that particular culture or, in case that person-specific information is available, for that specific person;
- Observe what the person does and says during the interaction, acquiring verbal, and nonverbal inputs through robot sensors;
- Revise with cultural sensitivity the knowledge that the robot has about the person, that is, by adequately updating “familiarity” values in the person-specific layer depending on what the person said in reply to the robot and on the observed behavior of the person;
- Evaluate, together with the person, the robot’s behavior, that will enable the system to improve in time.

We do not provide additional details on how the ADORE model is implemented as a computer program. Still, please remember that the Ontology is just knowledge structured in a particular way, that is, something static: to use such knowledge according to the steps required by the ADORE model, a computer program needs to be implemented that repeatedly acquires data from the environment through sensors, processes them at the light of the available cultural knowledge, perform actions in the environment in a way that is appropriate depending on the information in the culture-specific and the person-specific layers, and possibly update the knowledge in the ontology accordingly. The program to implement the ADORE guidelines will have the structure, generally speaking, of the flowcharts shown in [Chapter 2](#): we report an example in [Fig. 8.5](#).

8.3.2 From knowledge representation to language generation and dialog management

As mentioned in the previous section, another essential data property is associated with concepts, allowing experts to write sentences to talk about them. When interpreting concepts as something that one can speak of, we refer to them as “topics of conversation” or simply

“topics.” There are as many topics in the Ontology as there are concepts, and every concept is a possible topic of conversation as long as it has sentences associated with it. Topics of conversation that are more or less relevant for different cultures may, once again, be determined by experts—see [Chapter 7](#).

Please notice that the sentences associated with topics may have different roles: they can be used by the robot to ask questions to the user, which can either be yes/no questions (e.g., “Do you ever have espresso in the morning?”) or open ones (e.g., “Please tell me what you like about having a coffee with friends”), they can be positive assertions about some topic (e.g., “I think that having an espresso is the best way to taste coffee”) or negative ones (e.g., “I don’t like espresso, there is nothing to drink there!”). Also, the same topic may have more than one sentence of the same kind associated with it: this means that the robot will have different possibilities to choose among when asking a question or making a positive or negative assertion. Finally, the same sentence can be stored in multiple languages to allow the robot to talk with people of different nationalities.

This approach might raise some criticism: we are in the Era of neural networks: as discussed in [Chapter 2](#), the so-called neural generative models ([Wang et al., 2017](#)) can be used to produce faked data that look realistic starting from examples and can be used to synthesize everything from pictures to music, to alter a video of a trotting horse to a trotting zebra, to switch the face of Jack Nicholson in Kubrick’s *Shining* with Jim Carrey’s⁴, or more simply to change the age and gender of a person in a photograph (we suggest to try FaceApp⁵ if you haven’t done it yet). Given that generative models can learn how to do or say things starting from a large amount of available data, do we really need to store prewritten sentences in the Ontology to allow the robot to talk about the concepts that are encoded in it? Why not design a robot that learns how to talk about a topic by taking inspiration from discussions that can be found on the Internet, for example, in forums or social media? If this is possible, there would be no need to write or store any sentence in the Ontology manually: the robot will autonomously learn how to reply to what the user says, perhaps considering not only the last sentence of the user but the current context, that is, the history of the most recent things that the person has said. We expect the result of this process to be more natural than using prewritten sentences.

Neural generative models exist that automatically produce replies depending on what the user said. They are used by some of the most popular chatbots ([Zhou et al., 2020](#)) to make realistic discussions with users. However, the risk of generative models has already been mentioned in [Chapter 2](#). Who has control of what such a system will say? How can we ensure that the sentences in the dataset are appropriate for the user, that is, they handle sensitive matters in the right way, do not contain offensive languages, etc.? This is known to be an open problem of generative models: developers do not have control of what the system will say, which can be very dangerous, especially in sensitive contexts. Social and health care is precisely one of such contexts, which requires handling this issue with greater attention by guaranteeing that everything the robot says is appropriate for the context in which the interaction occurs.

Having abandoned the idea of using a neural generative model, we need another solution to guarantee sufficient variability in the robot’s sentences: as the reader can imagine, a whole Ontology that covers all aspects of daily life, including also the actions that the robot might

⁴ https://www.youtube.com/watch?v=HG_NZpkttXE

⁵ <https://www.faceapp.com/>

perform, may easily include more than 10,000 concepts. [Chapter 2](#) extensively debated the importance of “robust” systems capable of providing a coherent reply in almost all situations. Who will manually encode all the required sentences to talk about such topics of conversation, considering that each topic will likely have multiple sentences associated with it to allow a sufficient degree of variability when the robot asks questions or makes assertions?

The key idea is to exploit the hierarchy of the Ontology: that is, the fact that knowledge is hierarchically organized into concepts (e.g., “Coffee”), children of such concepts through a parent-child “is-a” relationship (e.g., “Espresso coffee is-a Coffee”) and other concepts related to them through a “has-N” property (e.g., “Coffee has-1 Brand”). Please remember that the hierarchical structure of the Ontology implies a side effect: if a concept has a property, this property is inherited by all children. In our system, inheritance turns out to be very useful for allowing children topics to inherit sentences from parent topics. Suppose that the topic “Hot drink,” which is the parent of “Coffee,” has been associated with the sentence “Great, I love to have a hot drink when I wake up from my bed”: the sentence will be inherited by “Coffee” and “Espresso coffee,” as well as by all the other children of “Hot drink” and the children of these children. Whenever the robot is talking about “Coffee,” “Tea,” “Espresso coffee,” or any other subconcept in the hierarchy, this sentence will be available and associated with that concept.

Let’s consider a possible dialogue about the robot and a person.

User: I love Coffee in the morning.

Robot: Great, I love to have a hot drink when I wake up from my bed.

User: I love to have Espresso Coffee, to be more precise.

Robot: Great, I love to have a hot drink when I wake up from my bed.

There are two evident problems in this exchange between the user and the robot.

First, we cannot use only one sentence for all the topics in the hierarchy: the final result sounds quite repetitive, doesn’t it? A solution to this problem is to add more than one sentence that can be used to make positive assertions: if we have 10 sentences for “Hot drink,” the chances of having a repetition are lower. But this is not the only problem.

The second problem is that the reply is not very specific. The user is talking about “Coffee” and about “Espresso coffee,” not about a generic “Hot drink.” The sentence contains the word “Hot drink” because the sentence was initially stored in the parent concept. However, in the second part of this short dialogue, we are not talking about “Hot drink,” but about its child “Coffee”: can’t the robot say something more specific?

The key, once again, is to exploit the inheritance mechanism of Ontologies. We can store in the Ontology sentences that contain variable parts: a sentence will be associated with a parent concept at the higher level of the hierarchy and inherited by children, but it will not be inherited “as is.” Instead, when inherited by a child topic, all variable parts of the sentences will assume a value that depends on the child inheriting it. For example, let’s consider again the sentence “I love to have *a hot drink* when I wake up from my bed”: the words “a hot drink” are now in *Italics*, which means they are a variable part of the sentence. When inherited by “Coffee,” the sentence will become “I love to have *a coffee* when I wake up from my bed”; when inherited by “Espresso coffee,” it will become “I love to have *an espresso*

when I wake up from my bed.” Suppose that the concept “Hot drink” has 50 subconcepts: by writing just one sentence, we will have added 50 sentences in the Ontology, each specifically tailored to talk about that topic.

The newly added sentences are more specific than the previous ones; however, this is not sufficient. All sentences inherited by subconcepts look very similar, and the reason is that we are only exploiting “is-a” parent–child relationships between concepts, but we are entirely ignoring “has-N” relationships relating a concept to its properties. Suppose that the concept “Food” is related to the concept “Time of the day” through a property “has-1-Time of the day,” Fig. 8.6, meaning that some food may be more or less common at breakfast, lunch or dinner. “Hot drink” is a child of “Food,” whereas “Breakfast time,” “Lunchtime,” “Dinner time” are all children of “Time of the day.” Then, we might think about storing the sentence “I usually have *food* *sometimes*” in the concept “Food,” where *food* and *sometimes* are in Italics and therefore are interpreted as parts of the sentence that may vary. As previously, the sentence will be automatically inherited by all children of “Food,” which will substitute the variable part *food* with the name of the inheriting concept. Additionally, the variable part *sometimes* will be substituted as well, but this time with the name of the concepts associated with “Food” and its children through the “has-1-Time of the day” property. By adding just one sentence written by hand in “Food,” this will produce a set of sentences that share a common structure but are indeed quite different from each other. Some examples are: “I usually have *cappuccino at lunchtime*”; “I usually have *cappuccino at breakfast*”; “I usually have *coffee after dinner*.” The number of automatically produced sentences can become huge when the number of children increases! Other variations of the same sentence can be produced by adding random prefixes or suffixes, such as “Do you know? I usually have Indian food at lunch”, “I need to tell you that I usually have Espresso at Breakfast. I am not the only one”, or “Yep, I usually have coffee after dinner. You should try.”

Finally, Fig. 8.6 shows sentences as if they are only encoded in the culture-generic TBox for ease of representation. However, it is also possible to encode sentences in culture-specific ABox, to express that different cultures are likely to have a different approach to talk about the same topic.

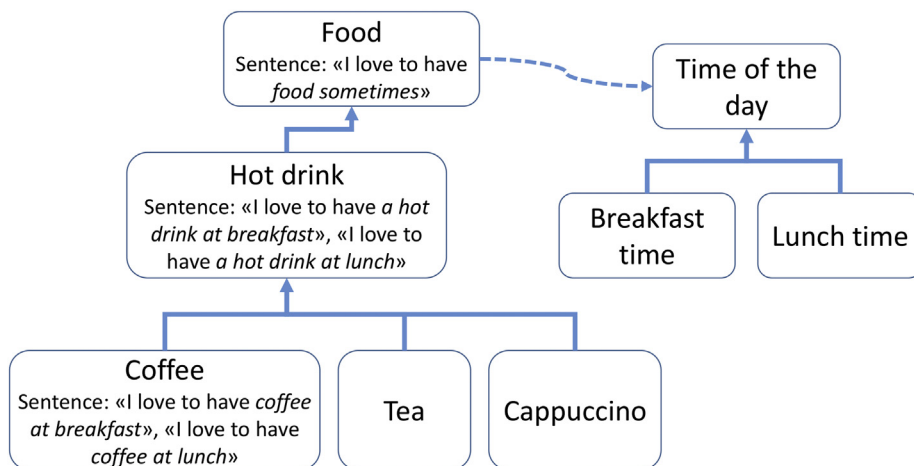


FIGURE 8.6 Concepts in the ontology with associated sentences.

8.3.3 From language generation to dialogue management

The mechanism above clarifies how sentences are automatically composed starting from sentences stored into topics of conversation, that is, concepts of the Ontology. Next, we need a procedure telling the robot when and how to talk with the user using such sentences.

If you refer to the ADORE program in Fig. 8.5, we are now focusing on the “Assess,” “Do,” and “Observe” blocks of the program. When focusing on actions that the robot can “physically do” to interact with the environment and the person (which are being paid lesser attention in this discussion), the blocks “Assess” and “Do” of the ADORE model refer to the physical actions that the robot is executing, and the block “Observe” refers to the observations that the robot is making through its sensors. When focusing on the verbal interaction between the robot and the person, the block “Assess” refers to choosing the correct culture-specific layer and topic of conversation, the block “Do” refers to selecting the new sentence to be pronounced among those stored in the Ontology and automatically composed, and the block “Observe” refers to the robot acquiring the reply of the person, interpreting it, and possibly using the information contained therein to select the next robot’s sentences.

You may know more about the implementation of the ADORE model as an AI program in Chapter 9. Here, we will focus on the implementation of the “Assess,” “Do,” and “Observe” blocks for culturally competent verbal conversation. This mechanism is sometimes referred to with a name: “chit-chatting.” Other researchers call this kind of verbal interaction “open conversation,” meaning a conversation that is not bounded to a specific application domain (Huang et al., 2020). In the case of the robot, chit-chatting is different from a task-oriented conversation, that is, issuing verbal commands to perform tasks, such as turning on the lights or playing a song. Instead, chit-chatting refers to a conversation open to any possible topic, without any additional purpose than the conversation itself. The distinction between chit-chatting and a task-oriented conversation should be straightforward: see two examples of task-based conversation (first dialog) and chit-chatting (second dialog) below to avoid confusion.

U: robot, tell me the weather.

R: For sure: tell me the location you want to know the weather about.

U: Rome.

R: Are you a religious person?

U: Well, I am not sure. Sometimes.

R: Believing in something may be important for people. Please, tell me more about what you think about this.

Since we emphasize chit-chatting, it is essential to clarify that task-based conversation is crucial as well: indeed, it is the basis for giving instructions to the robot to do things. Task-based conversation will not be discussed in detail only because it is less related to cultural competence: when we want to assign a task to the robot, we are willing to learn specific commands for the robot to understand us!

Ideally, task-based conversation and chit-chatting might even be performed by two completely distinct mechanisms. The interpretation of the user’s commands to assign tasks

to the robot may, for instance, be based on the detection of simple keywords in the user's speech or rely on a third-party system on the Internet for Natural Language Processing (NLP). There are many commercial systems available that work very well in the case of speech processing for command understanding: given a sentence such as "Robot, tell me the weather in London," these systems have no problems in extracting the command (i.e., "tell me the weather") and additional parameters (i.e., "in London"). On the contrary, the system for culturally competent chit-chatting works on different principles and cannot be easily implemented using off-the-shelf NLP tools. For this reason, our proposal to develop such a system is described in greater detail. Finally, remark that the command interpreter for task-based conversation and the procedure for chit-chatting shall always work in parallel: at any time during chit-chatting, the person might ask the robot to do something specific such as turning on the lights; vice versa, when the person issues a command that the robot is not capable of interpreting, the robot should at least be capable to chit-chat about it. See a possible conversation interweaving task-based conversation and chit-chatting below.

R: Are you a religious person? (chit-chatting about religion).
U: Well, I am not sure. Sometimes. (chit-chatting about religion).
R: Believing in something may be important for people. Please, tell me more about what you think about this. (chit-chatting about religion).
U: Ok, but before turn-on the air conditioner, please. It's too hot! (command that the robot can execute).
R: Ok, done. (command that the robot can understand).
U: Bring me a glass of water, please. (command that the robot cannot execute).
R: It's very important to drink water regularly. (chit-chatting about drinking water).

The requisites to allow successful chit-chatting with the user are different from task-based conversation. In chit-chatting, the key is not understanding what the user wants to do using advanced NLP techniques but rather managing the dialog with the user in a way that is engaging and rewarding by providing the robot with the opportunity to show its knowledge and cultural competence. Specifically, to design a system for culturally competent chit-chatting, two simple mechanisms might be implemented. Mechanism one and Mechanism two heavily rely on how we proposed to organize the knowledge in the Ontology.

Mechanism one is based on the fact that all topics in the Ontology are already related to each other through semantic relationships: parent-child "is-a" relationships between concepts encode the information that the parent concept is more general concept than the child concept, that is, "Espresso coffee is-a Coffee"; "has-N" properties relate concepts that are semantically close to each other in a given domain, that is, "Coffee has-1 Brand". So then, as far as links between concepts correspond to the notion of semantic closeness between topics of conversation, they may be used to explore the Ontology by guaranteeing a coherent conversation in which subsequent sentences will always be related to each other. Consider once again [Fig. 8.6](#), where the links between concepts identify semantic relationships are shown. The robot might start talking about "Coffee," next move to "Espresso," and finally to the "Brand" of espresso that the user prefers (remember that all "has-N" properties are inherited from parents to children in the Ontology, even if they are not shown in the figure).

This mechanism might produce, for instance, the following conversation.

R: Do you like coffee? (the topic is “Coffee”; Yes/no question).
 U: Yes, I definitely need coffee when I wake up. (the topic is “Coffee”)
 R: Great! I think that having a coffee in the morning can improve your day. Tell me what you like the most about coffee if you want. (the topic is “Coffee”; Positive Sentence followed by an open question).
 U: Well, I like that it really wakes you up, especially if you did not sleep too well. Something which happens very often in the last weeks. (the topic is “Coffee”)
 R: I understand. And do you like espresso coffee? (the topic is “Espresso coffee”; Yes/no question).
 U: Of course, when I say “coffee” I mean “espresso”, I am Italian. (the topic is “Espresso coffee”)
 R: Good to know. Espresso is very popular in Italy and France, but not everywhere in the world. Tell me what you like the most about Espresso Coffee if you want. (the topic is “Espresso coffee”; Positive Sentence followed by an open question).
 U: Well ... Not sure ... for me Espresso coffee is just ... coffee. I don't like American Coffee. (the topic is “Espresso coffee”)
 R: I see. Do you like Illy Coffee? (the topic is “Brand”; Yes/no question).
 U: No ... it's a little too expensive. (the topic is “Brand”)
 R: I see. I don't like that brand as well, it's too sour. (the topic is “Brand”; Negative Sentence).

The dialog above provides the opportunity to see Mechanism one at work. First, there are three topics of conversation in this short dialog: the abovementioned “Coffee,” “Espresso coffee,” and “Brand.” These topics are connected in the Ontology through “is-a” or “has-N” relationships. Since, for the moment, we are only considering Mechanism one to browse the knowledge base, the sequence in which different topics of conversation are addressed may look very rigid: when a topic of conversation is considered as fully covered, the robot will pick one of the connected topics randomly, with a probability that is proportional to the “familiarity” in the culture-specific layer (i.e., the “familiarity” that the user is expected to have toward that topic depending on his or her culture). For instance, in the dialogue above, after talking about “Coffee” the robot has chosen “Espresso Coffee”: this may be a consequence of the fact that the person interacting with the robot is Italian, and then “Espresso Coffee” has a very high probability of being chosen, or simply because all choices ultimately have a chance of being selected, and even a person that has a very low familiarity with “Espresso Coffee” has a nonzero probability that this topic will be chosen.

Second, every topic of conversation requires multiple exchanges between the robot and the user, with the robot using the sentences encoded in the Ontology to talk. Indeed, the dialog clarifies that the sentences stored in the Ontology may be of different kinds:

- Yes/no questions such as “Do you like coffee?”, to which the user can reply with “yes,” “no,” but also variations such as “sure,” “of course,” “always,” “never,” etc. Yes/no questions are essential because the person's reply is easily recognizable as either a positive or negative reply. Then, such a reply can be used to update “familiarity” values in the person-specific layer corresponding to that user. For example, remember that all Italian users are initially assumed to have a very high “familiarity” with “Espresso coffee,” but a specific user may not like it. User preferences can be unambiguously acquired

through a straightforward Yes/no question (and, consequently, the robot may choose to avoid that topic in the future).

- Positive sentences such as “Great! I think that having a coffee in the morning can improve your day.” The robot says positive sentences following an affirmative reply to a Yes/no question.
- Negative sentences such as “I don’t like that brand as well, it’s too sour.” The robot says negative sentences following a negative reply to a Yes/no question.
- Open questions such as “Tell me what you like the most about coffee if you want.” The robot asks open questions to allow the person to talk freely about a topic, thus encouraging him or her to say things that may be helpful to jump from a topic a conversation to another one. Jumping from one topic to another is dealt with by Mechanism two, which will be soon explained.
- Suggestions for activities, such as “Do you want me to order coffee from the online grocery?” These sentences may or may not be added to some topics to trigger activities. There is no such example in the dialog above, but this and other kinds of sentences may be added to the Ontology structure to make the conversation around topics richer and more engaging.

Mechanism one may not be sufficient to guarantee a sufficiently rich and engaging interaction with the person. For this reason, Mechanism two is implemented, allowing to break this predefined dialog pattern and explore the Ontology unpredictably depending on what the user says.

Mechanism two can be as simple as a keyword-based mechanism detecting keywords in what the person says, or it can be based on more sophisticated NLP tools capable of understanding, in a general sense, the topic about which the person is talking. For instance, a simple keyword-based mechanism might extract the keywords “football” and “TV” from the sentence “On Sunday evening, I love watching the football match on TV in the common room.” In contrast, a more complex service for NLP running on the Internet may be capable of understanding that this sentence is about sport, entertainment, television, and things alike. Whatever strategy is used to detect the topic of conversation (possibly both), the information is then used by Mechanism two for switching in-between topics of conversation, thus breaking the rigid pattern determined by Mechanism one.

Consider the same dialogue as before, but now assume that Mechanism two is also active: the following exchange may be produced.

R: Do you like coffee? (the topic is “Coffee”; Yes/no question).

U: Yes, I definitely need coffee when I wake up. (“wake up” is recognized as a keyword).

R: Do you like waking up in the morning? (the topic is “Waking up”; Yes/no question).

U: Yes, I like it if it is a sunny day.

R: Great to know. Many people like to wake up early in the morning, and I think this is good for health, especially after a good sleep! Tell me more about what you like about waking up early in the morning. (the topic is “Waking up”; Positive Sentence followed by an open question).

U: Well, most of all, I like having a cup of coffee waiting for me! (“coffee” is recognized as a keyword).

R: Great! I think that having a coffee in the morning can improve your day. (the topic is “Coffee”; Positive Sentence followed by an open question).

It is essential to be aware that, during the conversation, the robot is not capturing all details of what the person is saying. Understanding everything is not the purpose of chit-chatting, and achieving this would be highly complex considering the vast range of topics that can be raised during a conversation. However, the robot can understand when the person wants to move from one topic to another. In the example, it promptly switches from “Coffee” to “Waking up” and then back to “Coffee,” following the rationale of what the person says.

This approach is pretty simple; however, it can “almost always” pick a coherent reply. It starts from the consideration that, at the end of the day, the most frustrating response you may get from a robot during the conversation is probably “Sorry, I don’t understand what you are talking about.”

Isn’t it?

8.3.4 Acquiring new cultural knowledge

The system described so far uses the information in the Ontology, particularly the “familiarity” data property in the culture-specific layer, to make educated guesses about what the person may like or not like depending on a priori knowledge about his or her culture. During the conversation, the person is allowed to express his or her actual preferences, which will be used to update the knowledge about that person in the person-specific layer of the Ontology. Next, person-specific knowledge may be used to drive the conversation toward topics that the person likes instead of using prestored information. This mechanism allows for avoiding stereotyped representations of people and their cultures; however, it is possible to do more.

It is reasonable to conjecture that there may be a correlation between the “familiarity” that a person has with a concept, that is, “watching tennis on TV,” and his or her “familiarity” with another concept, that is, “playing tennis.” Of course, a rigid causal dependence between playing tennis and watching tennis on TV cannot be inferred. However, there are reasonable elements to think that a person who likes to play tennis (or played tennis when he or she was young) is more willing to watch tennis on TV. Similarly, think about an English person that likes Japanese paintings, writes haiku poems, and loves sushi. Wouldn’t you be tempted to guess that this person, even if she is not from Japan, is attracted by the Japanese culture? Maybe this person would be more willing than another one to visit Japan? Can we infer that this person would be more interested in talking about kimono manufacturing or zen gardens than a person that hates Japanese paintings, does not understand what’s the point in reading a haiku, and thinks that sushi is “just” raw fish?

Our cultural preferences are related to each other, possibly because they may have a common cause: perhaps the first person has a Japanese friend or watched Japanese animated movies when a child. Causal relationships are challenging to prove; this is a general truth in scientific research. However, correlations between facts in a probabilistic sense are much easier to find, even if they may sometimes lead to wrong conclusions. Can we make inferences about the unknown preferences of a person based on what we already know about him or her? The reply is yes: this kind of inference, which is necessarily probabilistic in its nature since a cause-effect relationship may not always exist, can rely on the use of Bayesian networks that have been introduced in [Section 7.2](#).

As usual, the familiarity associated with the concept “Zen garden” in the culture-specific layer corresponding to the English culture may be interpreted as a probability: the probability that a randomly chosen English person, if asked, will reply that he or she is familiar with zen gardens. Suppose that the “familiarity” with “Zen garden” has also been computed for the culture-specific layer corresponding to the Italian and Japanese cultures. If you consider all these “familiarity” values taken together, it is pretty straightforward to interpret them as a set of conditioned probabilities. More specifically, the three “familiarity” values associated with “Zen garden” and corresponding to the English, Italian, and Japanese culture-specific layers can be interpreted as “the conditioned probability that a person is familiar with zen gardens given that he or she is English”; “the conditioned probability that a person is familiar with zen gardens given that he or she is Italian”; “the conditioned probability that a person is familiar with zen gardens given that he or she is Japanese.” After some additional considerations that would require technical details beyond the purpose of this book, it may be shown that all these conditioned probabilities may be used to build a Bayesian Network that mirrors the Ontology structure (Bruno et al., 2019).

A figure of this new Bayesian Network is not reported; however, please consider once again Fig. 8.2, where a Bayesian Network was shown containing all the required information to infer, for instance, the probability that it is raining given that the grass is wet. Analogously to what happened with that Bayesian Network, the robot may use the Bayesian Network that mirrors the Ontology structure to make inferences about hidden facts starting from observed facts. For example, suppose that, at some point, the robot acquires information about the person, that is, the fact that he or she has familiarity with Japanese painting and poetry (observed facts). Then, by using the structure of the network and all the conditioned probabilities, the robot will infer that the probability that this person is familiar with other aspects of the Japanese culture (hidden fact) has increased and will use this information during the interaction. For instance, to guess other Japanese things that the person may be interested in, for example, zen gardens, or propose related activities, for example, suggesting an online shop to buy a kimono.

In conclusion, Bayesian Networks reveals additional tools that may parallel Ontologies to enable a proper culturally competent behavior. As soon as the system “observes” more and more about the person, a complex personalized representation of the user will emerge in the person-specific layer, merging in the same person different aspects of different cultures, which is coherent with what happens with real people living in the real world.

8.4 Conclusions

Culturally competent robots need cultural knowledge to interact appropriately with people; however, this knowledge must be encoded in a formalism that a robot can process and understand. To this end, we introduced two of the most popular approaches to encode knowledge in a formalism easily understandable both by humans and robots, Ontologies and Bayesian Networks. Next, we discussed the basic principles for designing a computer program that uses such knowledge to implement the guidelines provided by transcultural nursing experts. During this process, the readers should have familiarized themselves with the mental approach that computer scientists adopt to develop complex AI and Robotic solutions and become aware of the vast amount of work required when pursuing the goal of culturally competent robotic behavior.

8.4.1 Reflective questions

- When we use an ontology to describe a part of the world that is interesting for a computer application, we say that we are building a world model. What does it mean? What are the choices that Computer Scientists need to make when building such a model?
- What is the characteristic of computer ontologies that can be exploited to represent different views of the world corresponding to multiple cultures? How can this characteristic be used to avoid a stereotyped representation of people and cultures?
- Bayesian networks allow computing the probability of a fact or event we do not know based on what we know. How can this property be exploited to have a more faithful representation of people's attitudes toward things?

References

- Baader, F., Calvanese, D., McGuinness, D., Nardi, D., & Patel-Schneider, P. (Eds.). (2007). *The description logic handbook: Theory, implementation and applications* (2nd ed.). Cambridge: Cambridge University Press. <https://doi.org/10.1017/CBO9780511711787>
- Bruno, B., Recchiuto, C. T., Papadopoulos, I., Saffiotti, A., Koulouglioti, C., Menicatti, R., Mastrogiovanni, F., Zaccaria, R., & Sgorbissa, A. (2019). Knowledge representation for culturally competent personal robots: Requirements, design principles, implementation, and assessment. *International Journal of Social Robotics*, 11(3), 515–538.
- Carrithers, M., Candea, M., Sykes, K., Holbraad, M., & Venkatesan, S. (2010). Ontology is just another word for culture: Motion tabled at the 2008 meeting of the group for debates in anthropological theory, University of Manchester. *Critique of Anthropology*, 30(2), 152–200.
- Cimiano, P. (2006). Ontology learning and population from text: Algorithms, evaluation and applications. *Ontology Learning and Population from Text: Algorithms, Evaluation and Applications*, 1–347.
- Grau, B. C., Horrocks, I., Motik, B., Parsia, B., Patel-Schneider, P., & Sattler, U. (2008). Owl 2: The next step for OWL. *Web Semantics*, 6(4), 309–322.
- Gruber, T. R. (1995). Toward principles for the design of ontologies used for knowledge sharing. *International Journal of Human-Computer Studies*, 43(5–6), 907–928.
- Henare, A., Holbraad, M., & Wastell, S. (2006). Thinking through things: theorising artefacts ethnographically. *Thinking Through Things: Theorising Artefacts Ethnographically*, 1–237.
- Huang, M., Zhu, X., & Gao, J. (2020). Challenges in building intelligent open-domain dialog systems. *ACM Transactions on Information Systems*, 38(3).
- Pearl, Judea (2000). *Causality: Models, reasoning, and inference*. Cambridge University Press, ISBN 978-0-521-77362-1.
- Schmidhuber, J. (2015). Deep learning in neural networks: An overview. *Neural Networks*, 61, 85–117.
- Wang, K., Gou, C., Duan, Y., Lin, Y., Zheng, X., & Wang, F.-Y. (2017). Generative adversarial networks: Introduction and outlook. *IEEE/CAA Journal of Automatica Sinica*, 4(4), 588–598, 8039016.
- Zhou, L., Gao, J., Li, D., & Shum, H.-Y. (2020). The design and implementation of xiaoice, an empathetic social chatbot. *Computational Linguistics*, 46(1), 53–93.